Day 11: React Components, State & Forms

Student Registration Form in React

1. Topics Covered

- Functional components
- Props and state
- useState hook
- Event handling
- Controlled components
- Form handling
- Conditional rendering
- Basic form validation

2. Task

- Create a student registration form in React.
- Capture student name, email, and course.
- Display submitted details on the screen.
- Add basic validation.
- Use state to update the UI dynamically.

Expected output: a React student registration form with working form validation and state management.

3. Concepts Explained

Functional component and props. A functional component is a JavaScript function that returns JSX. Props are the inputs passed to it, like function arguments, and they are read-only. In this project `Field` and `StudentCard` receive their data through props.

State and useState. State is data that belongs to a component and changes over time. `const [value, setValue] = useState(initial)` returns the current value and a function to update it. Calling the setter makes React re-render the component with the new value.

Controlled components. An input is controlled when its value comes from state (`value={values.name}`) and every keystroke updates that state through `onChange`. React state is then the single source of truth for the form.

Event handling. Handlers such as `onChange`, `onBlur` and `onSubmit` respond to user actions. `e.preventDefault()` stops the browser from reloading the page when the form is submitted.

Conditional rendering. JSX shows or hides parts of the UI based on state, using `condition && <Element/>` or a ternary `condition ? A : B`.

4. How the Project Works

State variable	Purpose
values	Current text of the name, email and course inputs (controlled inputs).
errors	Validation messages produced by the validate() function.
touched	Which fields the user has visited, so errors appear only after leaving a field.
students	List of successfully registered students shown in the right-hand panel.
success	Message shown in a green banner after a valid submission.

Validation rules

Field	Rule	Error message
Full name	Required, at least 3 characters	Enter the student's full name. / Name must be at least 3 characters.
Email	Required, must match name@domain.ext	Enter an email address. / Enter a valid email, like name@example.com.
Course	Must select one option	Choose a course.

Flow of a submission

- The user types. `handleChange` updates `values`, and re-validates if the field was already touched.
- The user leaves a field. `handleBlur` marks it touched and shows its error, if any.
- The user clicks Register student. `handleSubmit` validates every field. If any error exists, it stops and shows the errors.
- If everything is valid, the student is added to `students`, the success banner appears and the form is reset.
- The right-hand panel shows an empty-state message until the first student is added, then the list with a live count.

5. How to Run

Requires Node.js. Check with `node -v`.

```
cd C:\dev
mkdir student-app
cd student-app
mkdir src

npm init -y
npm pkg set type=module
npm pkg set scripts.dev=vite
npm install react react-dom
npm install -D vite @vitejs/plugin-react
```

Create these three files, then save the component below as `src/StudentRegistration.jsx`.

index.html

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Student Registration</title>
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.jsx"></script>
  </body>
</html>
```

vite.config.js

```
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'

export default defineConfig({ plugins: [react()] })
```

src/main.jsx

```
import { createRoot } from 'react-dom/client'
import StudentRegistration from './StudentRegistration'

createRoot(document.getElementById('root')).render(<StudentRegistration />)
```

Start the app with `npm run dev` and open `http://localhost:5173`.

6. Source Code: src/StudentRegistration.jsx

```
import { useState } from "react";

const COURSES = ["Web Development", "Data Science", "UI/UX Design", "Cyber Security"];

const initialValues = { name: "", email: "", course: "" };

// Pure validation function: takes form values, returns an errors object
function validate({ name, email, course }) {
  const errors = {};
  if (!name.trim()) errors.name = "Enter the student's full name.";
  else if (name.trim().length < 3) errors.name = "Name must be at least 3 characters.";

  if (!email.trim()) errors.email = "Enter an email address.";
  else if (!/^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(email)) errors.email = "Enter a valid email,
      like name@example.com.";

  if (!course) errors.course = "Choose a course.";
  return errors;
}

// Reusable child component: receives data through props
function Field({ label, id, error, children }) {
  return (
    <div className="field">
      <label htmlFor={id}>{label}</label>
      {children}
      {error && <p className="error" role="alert">{error}</p>}
    </div>
  );
}

// Child component to display a submitted student (props in action)
function StudentCard({ student, index }) {
  return (
    <li className="card">
      <span className="badge">{index + 1}</span>
      <div>
        <strong>{student.name}</strong>
        <div>{student.email}</div>
        <div className="muted">{student.course}</div>
      </div>
    </li>
  );
}

export default function StudentRegistration() {
  const [values, setValues] = useState(initialValues); // controlled inputs
  const [errors, setErrors] = useState({});            // validation messages
  const [touched, setTouched] = useState({});          // which fields were visited
  const [students, setStudents] = useState([]);        // submitted students
  const [success, setSuccess] = useState("");          // success banner

  // One handler for every input: updates the field and re-validates live
  const handleChange = (e) => {
    const { name, value } = e.target;
    const next = { ...values, [name]: value };
    setValues(next);
    setSuccess("");
    if (touched[name]) setErrors(validate(next));
  };

  // Show an error only after the user leaves a field
  const handleBlur = (e) => {
    const { name } = e.target;
    setTouched({ ...touched, [name]: true });
    setErrors(validate(values));
  };

  const handleSubmit = (e) => {
    e.preventDefault();
    const found = validate(values);
    setErrors(found);
    setTouched({ name: true, email: true, course: true });
    if (Object.keys(found).length > 0) return; // stop if invalid

    setStudents([...students, { ...values, name: values.name.trim() }]);
```

```
    setSuccess(`${values.name.trim()} is registered for ${values.course}.`);
    setValues(initialValues);
    setTouched({});
    setErrors({});
  };

  const show = (field) => (touched[field] ? errors[field] : "");

  return (
    <div className="page">
      <style>{css}</style>

      <main className="layout">
        <section className="panel">
          <h1>Student registration</h1>
          <p className="muted">Fill in the details below to enrol a student.</p>

          {/* Conditional rendering: banner appears only after a successful submit */}
          {success && <div className="success" role="status">{success}</div>}

          <form onSubmit={handleSubmit} noValidate>
            <Field label="Full name" id="name" error={show("name")}>
              <input
                id="name" name="name" type="text" placeholder="Aarav Sharma"
                value={values.name} onChange={handleChange} onBlur={handleBlur}
                className={show("name") ? "invalid" : ""}
              />
            </Field>

            <Field label="Email" id="email" error={show("email")}>
              <input
                id="email" name="email" type="email" placeholder="aarav@example.com"
                value={values.email} onChange={handleChange} onBlur={handleBlur}
                className={show("email") ? "invalid" : ""}
              />
            </Field>

            <Field label="Course" id="course" error={show("course")}>
              <select
                id="course" name="course"
                value={values.course} onChange={handleChange} onBlur={handleBlur}
                className={show("course") ? "invalid" : ""}
              >
                <option value="">Select a course</option>
                {COURSES.map((c) => <option key={c} value={c}>{c}</option>)}
              </select>
            </Field>

            <button type="submit">Register student</button>
          </form>
        </section>

        <section className="panel">
          <h2>Registered students ({students.length})</h2>
          {/* Conditional rendering: empty state vs. list */}
          {students.length === 0 ? (
            <p className="muted">No students yet. Submit the form and they will appear here.</p>
          ) : (
            <ul className="list">
              {students.map((s, i) => <StudentCard key={i} student={s} index={i} />)}
            </ul>
          )}
        </section>
      </main>
    </div>
  );
}

const css = `
  .page { min-height: 100vh; background: #eef2f6; padding: 32px 16px; font-family: system-ui,
      sans-serif; color: #1c2733; }
  .layout { max-width: 880px; margin: 0 auto; display: grid; gap: 20px; grid-template-columns:
      repeat(auto-fit, minmax(300px, 1fr)); align-items: start; }
  .panel { background: #fff; border-radius: 12px; padding: 24px; border: 1px solid #d7dee6; }
  h1 { font-size: 24px; margin: 0 0 4px; }
  h2 { font-size: 18px; margin: 0 0 12px; }
  .muted { color: #5b6b7b; font-size: 14px; margin: 0 0 16px; }
  .field { margin-bottom: 16px; }
  label { display: block; font-size: 14px; font-weight: 600; margin-bottom: 6px; }
```

```
  input, select { width: 100%; box-sizing: border-box; padding: 10px 12px; font-size: 15px;
      border: 1px solid #b8c4d0; border-radius: 8px; background: #fff; }
  input:focus, select:focus, button:focus-visible { outline: 3px solid #9cc4f5; outline-
      offset: 1px; }
  .invalid { border-color: #c0392b; }
  .error { color: #c0392b; font-size: 13px; margin: 6px 0 0; }
  button { width: 100%; padding: 11px; font-size: 15px; font-weight: 600; color: #fff;
      background: #1f5fa8; border: 0; border-radius: 8px; cursor: pointer; }
  button:hover { background: #174b87; }
  .success { background: #e3f4e8; color: #1e6b3a; padding: 10px 12px; border-radius: 8px;
      margin-bottom: 16px; font-size: 14px; }
  .list { list-style: none; padding: 0; margin: 0; display: grid; gap: 10px; }
  .card { display: flex; gap: 12px; padding: 12px; border: 1px solid #d7dee6; border-radius:
      8px; font-size: 14px; }
  .card .muted { margin: 2px 0 0; }
  .badge { flex: none; width: 28px; height: 28px; border-radius: 50%; background: #1f5fa8;
      color: #fff; display: grid; place-items: center; font-size: 13px; font-weight: 600; }
`;
```

7. Testing Checklist

- Click into a field and leave it empty: an error appears under that field.
- Enter an invalid email such as `abc`: the format error appears.
- Click Register student with an empty form: all three errors appear and nothing is added.
- Fill in valid details and submit: the success banner appears, the student is added and the form clears.
- Submit several students: the list and the count in the heading update each time.

8. Result

The form captures name, email and course through controlled inputs, validates them, stores submitted students in state and displays them on screen, with the UI updating dynamically after every action.